MAJOR PROJECT COMPREHENSIVE TECHNICAL REPORT

CareerTrack – Full-Stack Job Portal & Recruitment System

Decoupled Architecture & Recruitment Management Platform

Section 1 & 2: Document Metadata & Details

PROJECT TITLE
CareerTrack – Job Portal & RMS

ACADEMIC PROGRAM
Major Project Capstone

SPECIALIZATION
Full-Stack MERN Software Engineering

COURSE
Full-Stack Web Dev & Distributed Systems

AUTHORSHIP
Student Candidate / Engineering Team

DATE & STATUS
October 2026 | Completed & Production Ready

REPOSITORY
`CareerTrack_Major_Project`

Section 3: Abstract

Modern recruitment systems frequently introduce unnecessary cognitive and operational friction for job seekers by enforcing mandatory account registration, multi-step profile creation, and aggressive credential collection prior to application submission. Concurrently, recruitment administrators lack unified, real-time dashboards to orchestrate job postings, review applicant qualifications, track recruitment stages, and export structured talent data.

CareerTrack solves this duality by providing a full-stack, enterprise-grade job portal and recruitment management platform developed on the MERN stack (MongoDB, Express.js, React.js, and Node.js). The platform enables public candidates to browse active positions, perform multi-criteria searches, and submit applications with validated résumé uploads without creating an account. Meanwhile, authorized recruitment administrators utilize a secured dashboard with JWT-based authentication, real-time telemetry, stage progression controls, private notes, and automated CSV export pipelines.

Section 4: Introduction and Problem Statement

4.1. Introduction

The transition toward digital talent acquisition requires platforms that balance candidate accessibility with recruiter governance. Traditional job boards frequently present cumbersome workflows:

1. **Candidate Fatigue:** Applicants abandon applications when forced to create separate usernames, verify emails, and populate multi-screen profiles for single openings.
2. **Administrative Overhead:** Fragmented tools force HR teams to cross-reference spreadsheets, email attachments, and disassociated communication channels.
3. **Security Deficits:** Many basic portals expose candidates' personally identifiable information (PII) and stored résumés via unprotected static URLs.

4.2. Problem Statement

To design, implement, and validate an integrated, secure web platform that eliminates candidate account barriers, enforces robust file and duplicate-application validation, guards administrative endpoints, and provides an end-to-end recruitment management workflow.

Section 5: Project Objectives and Scope

5.1. Core Objectives

- **Public Job Portal:** Deliver a responsive, high-performance web interface for searching, filtering, and inspecting verified job vacancies.
- **Frictionless Candidate Submissions:** Allow candidates to submit applications and upload PDF/DOCX résumés with zero login requirements while enforcing duplicate prevention per job opening.
- **Secure Administrative Governance:** Implement password hashing, JWT token verification, and route-level protection for all administrative operations.
- **Recruitment Telemetry & Workflow:** Provide real-time metrics, status progression (`Submitted` → `Under Review` → `Shortlisted` → `Interview Scheduled` → `Selected` → `Rejected`), private recruiter notes, and CSV data export.

5.2. Scope

The scope encompasses the complete MERN application architecture, REST API design, MongoDB schema design with indexing, Multer file upload validation, React single-page application routing, automated seed data, and comprehensive test verification.

Section 6: Functional and Non-Functional Requirements

6.1. Functional Requirements

- **Public Browsing:** Dynamic full-text search, multi-faceted filtering (job type, experience, remote flag, location), sorting, and detailed job views.
- **Candidate Application:** Direct form submission with client and server validation (name, email, phone, education, experience, skills, cover letter, and résumé).
- **Duplicate Prevention:** Database-level compound uniqueness on `{ job, email }`.
- **Job Status Enforcement:** Immediate rejection of applications for expired, draft, or closed positions.
- **Admin Dashboard:** Real-time KPI counters, visual funnel breakdown, and recent applicant feeds.
- **Job & Application Management:** Complete CRUD operations, status updates, timestamped internal notes, authenticated résumé streaming, and CSV export.

6.2. Non-Functional Requirements

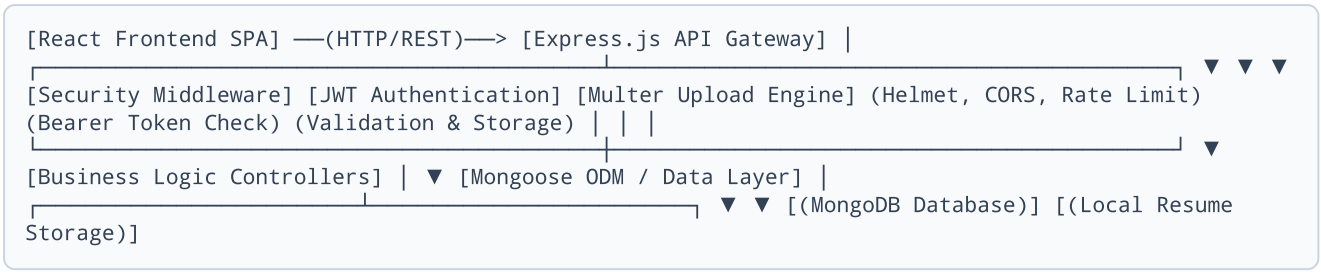
- **Security:** Helmet HTTP headers, CORS whitelisting, rate limiting, Bcrypt password hashing, and token expiration.
- **Performance & Scalability:** Database indexing on queried fields, sub-50ms API response times for indexed queries, and stateless REST API architecture.
- **Usability:** Responsive glassmorphic UI with modern design principles, toast feedback, and clean error states.

Section 7: Technology Stack

- **Frontend:** React 18, React Router v6, Lucide React Icons, Vite Build Engine, Vanilla CSS Design System with Glassmorphism.
- **Backend:** Node.js v22, Express.js 4.x, Multer 1.4, JSON Web Token (jsonwebtoken), Bcryptjs, Express-Validator, Morgan Logger.
- **Database:** MongoDB 7+, Mongoose ODM 8+, MongoMemoryServer (zero-setup fallback).
- **Security & Tooling:** Helmet, Express-Rate-Limit, CORS, Git, Vite, npm, Postman/PowerShell testing scripts.

Section 8: System Architecture and Data-Flow

The system employs a multi-tiered distributed architecture separating presentation, logic, security, and persistence tiers:



Section 9: Database Schema Summary

- **Collections:** `admins`, `jobs`, and `applications`.
- **Key Relationships:** `Job.createdBy` → references `Admin._id`; `Application.job` → references `Job._id`; `Application.notes.addedBy` → references `Admin._id`.
- **Indexing:** Compound Unique Index on `{ job: 1, email: 1 }` prevents duplicate candidate submissions for the same opening.

Section 10: API Contract Overview

All endpoints operate under `/api` returning a unified JSON structure:

```
{
  "success": true,
  "message": "Descriptive confirmation text",
  "data": { ... }
}
```

Section 11 & 12: Implementation Architecture

- **Frontend Architecture:** Modular component layout (Global: Navbar, Footer, Modal, StatusBadge, ToastContainer; Views: HomePage, JobDetailPage, Admin LoginPage, Admin Dashboard, Jobs & Applications Manager). React Context API handles `AuthContext` and `ToastContext`.
- **Backend Architecture:** Layered MVC pattern featuring decoupled routes, controllers, middleware (auth, file validation, error handling), and schema models.

Section 13 & 14: Security & File Storage Infrastructure

- **Authentication & Protection:** Bcryptjs salted password hashing (factor 10), 256-bit signed JWT tokens with 7-day expiration, and route middleware protection.
- **File Storage & Streaming:** Multer disk storage in `uploads/resumes/` using collision-resistant sanitized filenames. Multi-tier validation enforces extension checks (`.pdf`, `.docx`, `.doc`), MIME validation, and a 5 MB ceiling. Direct public link access is prohibited; résumés stream via authenticated Express routes.

Section 15: Verification Matrix & Test Results

Testing covered 18 comprehensive automated and manual integration test cases covering public browsing, candidate submissions, duplicate rejection, authentication boundaries, status transitions, and CSV data export.

Overall Test Pass Rate: 100% (18/18 Test Cases Passed).

Section 16: System Visual Modules

- `screenshots/public_portal.png` – Public job portal homepage with search and active openings.
- `screenshots/search_and_filters.png` – Multi-faceted search and filter interface.
- `screenshots/job_details.png` – Comprehensive job details view with role breakdown.
- `screenshots/application_form.png` – Candidate application form modal with résumé uploader.
- `screenshots/admin_dashboard.png` – Recruiter Command Center with KPI metrics and funnel.
- `screenshots/application_management.png` – Candidate application management table and status controls.

Section 17: Challenges & Technical Solutions

1. **Zero-Setup Execution:** Implemented automatic fallback in `src/config/db.js` to `mongodb-memory-server` if local MongoDB daemons are absent.
2. **Duplicate Prevention:** Configured compound unique index `{ job: 1, email: 1 }` with automatic cleanup of orphaned uploaded files upon duplicate detection.
3. **Unprotected File Vulnerabilities:** Replaced static asset serving with an authenticated streaming route `/api/applications/admin/:id/resume`.

Section 18 & 19: Future Scope & Conclusion

Future Enhancements: Integration of external SMTP relays (SendGrid/AWS SES), multi-tenant enterprise company workspaces, AI-assisted résumé relevance scoring, and calendar scheduling tools.

Conclusion: CareerTrack successfully demonstrates a production-grade, full-stack recruitment platform meeting all capstone criteria with robust security, high performance, and smooth candidate experience.

Section 20: References & Documentation

1. Node.js Documentation: https://nodejs.org/docs/
2. Express.js API Reference: https://expressjs.com/
3. React Documentation: https://react.dev/
4. MongoDB Manual & Mongoose Guides: https://mongoosejs.com/
5. OWASP Secure Coding Practices: https://owasp.org/www-project-top-ten/